

HDOC Day_27 Programming in C

Problem

Write a C program that displays the following menu and uses a switch statement to print the “Pyramid/Diamond pattern”

selected by the user. Use nested loops without arrays. Each pattern may be implemented in a separate parameterless

function with no return value. Note: For each activity, prepare the algorithm and test-case table. Then write, compile

and execute the C program, and test it against the prepared test cases.

1. Full pyramid

Input the number of rows, n, and print a full pyramid using asterisks. For n = 4:

```
  *
 * * *
* * * * *
* * * * * * *
```

2. Inverted pyramid

Input the number of rows, n, and print an inverted pyramid using asterisks. For n = 4:

```
* * * * *
 * * * *
  * * *
   * *
```

3. Diamond pattern

Input the number of rows, n, for the upper half and print a diamond pattern using asterisks. The middle row contains

$(2n - 1)$ asterisks, and the complete diamond contains $(2n - 1)$ rows. For n = 4:

```
  *
 * * *
* * * * *
```

* * * * *

* * * *

* * *

*

4. Palindromic number pyramid

Input the number of rows, n. In each row, print natural numbers in increasing order from 1 to the row number and

then in decreasing order back to 1. For n = 4:

1

1 2 1

1 2 3 2 1

1 2 3 4 3 2 1

5. Repeated-number pyramid

Input the number of rows, n. In each row, print the row number $(2n - 1)$ times, where n denotes the current row

number. For n = 4:

1

2 2 2

3 3 3 3 3

4 4 4 4 4 4 4

Algorithm

Algorithm:

1. Start.
2. Read the number of rows n.
3. Calculate $z = (2 \times n) - 1$.
4. Set $x = (z + 1) / 2$.
5. Set $b = 1$.
6. Repeat for $i = 1$ to n:

- Repeat for $j = 1$ to z :
 - If $j == x$, print b stars.
 - Otherwise, print spaces.
 - Move to the next line.
 - Decrease x by 1.
 - Increase b by 2.
7. Stop.
-

2. Inverted Pyramid

Algorithm:

1. Start.
 2. Read the number of rows n .
 3. Calculate $z = (2 \times n) - 1$.
 4. Set $x = 1$.
 5. Set $y = z$.
 6. Repeat for $i = 1$ to n :
 - Repeat for $j = 1$ to z :
 - If $j \geq x$ and $j \leq y$, print $*$.
 - Otherwise, print spaces.
 - Move to the next line.
 - Increase x by 1.
 - Decrease y by 1.
 7. Stop.
-

3. Diamond Pattern

Algorithm:

1. Start.
2. Read the number of rows m .

3. Calculate $n = m - 1$.
 4. Calculate $z = (2 \times n) - 1$.
 5. Calculate $x = (z + 1) / 2$.
 6. Set $b = 1$.
 7. **Print the upper half:**
 - Repeat for $i = 1$ to n :
 - Print two spaces.
 - Repeat for $j = 1$ to z :
 - If $j == x$, print b stars.
 - Otherwise, print spaces.
 - Move to the next line.
 - Decrease x by 1.
 - Increase b by 2.
 8. **Print the middle line:**
 - Print $2 \times m - 1$ stars.
 9. **Print the lower half:**
 - Set $f = 1$.
 - Set $y = z$.
 - Repeat for $i = 1$ to n :
 - Print two spaces.
 - Repeat for $j = 1$ to z :
 - If $j \geq f$ and $j \leq y$, print $*$.
 - Otherwise, print spaces.
 - Move to the next line.
 - Increase f by 1.
 - Decrease y by 1.
 10. Stop.
-

4. Palindromic Number Pyramid

Algorithm:

1. Start.
2. Read the number of rows n .
3. Repeat for $i = 1$ to n :
 - Print $n - i$ spaces.
 - Print numbers from 1 to i .
 - Print numbers from $i - 1$ down to 1.
 - Move to the next line.
4. Stop.

Example:

```
1
121
12321
1234321
123454321
```

5. Repeated Number Pyramid

Algorithm:

1. Start.
2. Read the number of rows n .
3. Calculate $z = (2 \times n) - 1$.
4. Calculate $x = (z + 1) / 2$.
5. Set $b = 1$.
6. Repeat for $i = 1$ to n :
 - Repeat for $j = 1$ to z :
 - If $j == x$:
 - Repeat b times and print the current row number i .

- Otherwise, print spaces.
- Move to the next line.
- Decrease x by 1.
- Increase b by 2.

7. Stop.

Example for n = 5:

```

1
2 2 2
3 3 3 3 3
4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5

```

6. Main Function / Menu

Algorithm:

1. Start.
2. Display the pattern menu:
 - 1 → Full Pyramid
 - 2 → Inverted Pyramid
 - 3 → Diamond
 - 4 → Palindromic Number Pyramid
 - 5 → Repeated Number Pyramid
3. Read the user's choice ch.
4. Use a switch statement:
 - If ch = 1, call full_pyramid().
 - If ch = 2, call invert_pyramid().
 - If ch = 3, call diamond().
 - If ch = 4, call Palindromic_pyramid().
 - If ch = 5, call Repeated_pyramid().

- Otherwise, display "**Invalid choice**".

5. Stop.

C program code

```
#include <stdio.h>

void full_pyramid()
{
    int n;

    printf("enter the number of rows\n");

    scanf("%d",&n);

    int z=(2*n)-1;

    int x=(z+1)/2;

    int b=1;

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=z;j++)
        {
            if(x==j)
            {
                for(int a=1;a<=b;a++)
                {
                    printf("* ");
                }
            }
            else
            printf(" ");
        }
    }
}
```

```

        printf("\n");

        x--;

        b=b+2;
    }
}

void invert_pyramid()
{
    int n;

    printf("enter no of rows\n");

    if (scanf("%d", &n) != 1 || n < 1)
    {
        printf("Please enter a positive integer.\n");

        return 1;
    }

    int z=(2*n)-1;

    int x=1;

    int y=z;

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=z;j++)
        {
            if(j>=x && j<=y)

                printf("* ");

            else

                printf(" ");

        }

        printf("\n");

        x++;
    }
}

```

```

        y--;
    }
}

void diamond()
{
    int m;

    printf("enter no of rows\n");
    if (scanf("%d", &m) != 1 || m < 1)
    {
        printf("Please enter a positive integer.\n");
        return 1;
    }

    int n=m-1;

    int z=(2*n)-1;

    int x=(z+1)/2;

    int b=1;

    for(int i=1;i<=n;i++)
    {
        printf(" ");

        for(int j=1;j<=z;j++)
        {
            if(x==j)
            {
                for(int a=1;a<=b;a++)
                {
                    printf("* ");
                }
            }
        }
    }
}

```

```

        else
            printf(" ");
    }
    printf("\n");
    x--;
    b=b+2;
}
for(int i=1;i<=(2*m-1);i++)
{
    printf("* ");
}printf("\n");
int f=1;
int y=z;
for(int i=1;i<=n;i++)
{printf(" ");
    for(int j=1;j<=z;j++)
    {
        if(j>=f && j<=y)
            printf("* ");
        else
            printf(" ");
    }
    printf("\n");
    f++;
    y--;
}
}
void Palindromic_pyramid()

```

```

{
    int n;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {

        // Print spaces
        for (int j = 1; j <= n - i; j++) {
            printf(" ");
        }

        // Print increasing numbers
        for (int j = 1; j <= i; j++) {
            printf("%d", j);
        }

        // Print decreasing numbers
        for (int j = i - 1; j >= 1; j--) {
            printf("%d", j);
        }

        printf("\n");
    }
}

void Repeated_pyramid()
{
    int n;

```

```

printf("enter the number of rows\n");
scanf("%d",&n);
int z=(2*n)-1;
int x=(z+1)/2;
int b=1;
for(int i=1;i<=n;i++)
{
    for(int j=1;j<=z;j++)
    {
        if(x==j)
        {
            for(int a=1;a<=b;a++)
            {
                printf("%d ",i);
            }
        }
        else
            printf(" ");
    }
    printf("\n");
    x--;
    b=b+2;
}
}
int main()
{
    int ch;

    printf("choice 1 is to print a full pyramid using asterisks..\n");

```

```
printf("choice 2 is to print a ligned decreasing triangle using asterisks\n");
printf("choice 3 is to print a diamond pattern of astricks\n");
printf("choice 4 is to print a Palindromic number pyramid");
printf("choice 5 is to print a Repeated-number pyrami");
printf("enter your choice\n");
scanf("%d",&ch);
switch (ch)
{
    case 1:
    {
        full_pyramid();
        break;
    }
    case 2:
    {
        invert_pyramid();
        break;
    }
    case 3:
    {
        diamond();
        break;
    }
    case 4:
    {
        Palindromic_pyramid();
        break;
    }
}
```

```
case 5:
{
    Repeated_pyramid();
    break;
}
default:
{
    printf("invalid input\n");
    break;
}
}
return 0;
}
```